DOCUMENT

08

Mutation Testing Report

Before and after scores, survivors and fixes

1. What was measured and how

Mutation testing changes the code in small ways (flip `<=` to `<`, change a constant, drop a statement) and checks whether the tests notice. A mutant that no test fails on is a **survivor**: a place where the tests would not catch a real bug. The score is `(killed + timeout) / total`.

Setting	Value
Tool	mutmut 3.8.0
Scope	`backend/src/invoice_app/domain` (excluding the unrelated discount calculator, errors, repository protocol, status enum)
Tests used	`test_approval_service.py` and `test_approval_policy.py`
Environment	WSL Ubuntu 22.04 (mutmut 3 needs `fork()`), Python 3.10
Command	`wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh <label>`
Reports	`docs/mutation/baseline.txt`, `after-first-pass.txt`, `after.txt`, `after-solid-refactor.txt`

2. Scores

Run	Code under test	Total	Killed	Survived	Score
Before (baseline)	Python port, the 7 original tests, no approval limits yet	36	23	13	**63.9%**
First pass after the feature	feature plus the first 75 tests	85	69	16	81.2%
After (before the SOLID refactor)	feature plus survivor-driven tests	86	86	0	**100.0%**
After the SOLID refactor (final)	rules split into classes, injected collaborators	132	132	0	**100.0%**

Before

```
Mutation report: baseline
Tool: mutmut mutmut, version 3.8.0
Total mutants : 36
Killed        : 23
Survived      : 13
Mutation score: 63.9% (killed+timeout / total)
```

After

```
Mutation report: after
Tool: mutmut mutmut, version 3.8.0
Total mutants : 86
Killed        : 86
Timeout       : 0
Survived      : 0
Other         : 0
Mutation score: 100.0% (killed+timeout / total)
```

3. Reading the numbers fairly

- The mutant count rises from 36 to 86 because the new `approval_policy.py` and `approval.py` are inside the scope. The scope is the whole domain package in every run.
- The 100% describes the code as it stands now; it is not a claim that the baseline code was equally tested.
- 100% means no surviving mutant in this scope. It does not prove the absence of bugs, and it does not cover the API layer.

After the SOLID refactor

```
Mutation report: after-solid-refactor
Tool: mutmut mutmut, version 3.8.0
Total mutants : 132
Killed        : 132
Timeout       : 0
Survived      : 0
Other         : 0
Mutation score: 100.0% (killed+timeout / total)
```

The refactor first produced **one survivor** (132 mutants, 131 killed, 99.2%): in `SecondApproverMustBeDirectorRule`, `>` changed to `>=` at exactly 500,000.00. In the combined policy an earlier rule hides that difference, so no existing test noticed. Now that each rule is its own unit, it needs its own boundary test. Per-rule boundary tests were added and the score returned to 100%.

4. Survivors and how they were closed

The `mutation-analyst` role classified the 16 first-pass survivors. **None was an equivalent mutant**; all were missing tests.

Cause	What the tests missed	Fix
No-op fake `update`	The test fake did not store or record saves, so "forgot to save" passed	Fake now stores deep copies, records updates and raises on unknown ids; persistence tests added
Unasserted `approved_at`	The timestamp could be dropped or changed without a failing test	Tests assert each approval is stamped and in UTC
Naive timestamps	Mutating the clock helper to a naive datetime survived	Tests assert timezone-aware UTC
Unasserted messages	Reason and state-error texts could change silently	Exact message assertions everywhere

The survivors were about persistence, timestamps and message texts, not about the limit comparisons: the one-cent-either-side boundary tests were written from the start, and none of the 16 survivors was a boundary mutant.

5. Effort and lessons

- A test that only checks the exception type for a refusal kills almost nothing; asserting the reason and the unchanged invoice is what raised the score.
- A fake that is too forgiving hides bugs from mutation testing as well as from humans.
- The `test-reviewer` pass flagged the forgiving fake and the mutation run confirmed, in numbers, that it hid real gaps; the two techniques complement each other.
- Mutation testing is slow and Unix-only here, so it is its own CI job and is run on demand locally.

6. How to reproduce

```
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh after
wsl -d Ubuntu-22.04 -- bash scripts/show_survivors.sh
```

The script copies `backend/` to a native Linux directory (fast, avoids the OneDrive mount), runs `mutmut`, and writes the plain-text report to `docs/mutation/<label>.txt`.